Purpose - How to solve problems

In space and time…

Learning Goal

This part of the project aims to outline the general process of solving a problem in competitive programming so that the learner could solve simple problems (A+B, print `hello world` etc), and introduce general jargon that will be used in the field of competitive programming.

Purpose

The general purpose of solving a problem in competitive programming is as following.

1. You want to create a program that given any valid input, outputs the correct output given in the problem description.
2. Your program must run within a set time limit, specified in the problem description.
3. Your program must run within a set memory limit, also specified in the problem description.

In a typical competition or in an online judge (such as codeforces or leetcode), if your solution satistifies these three constraints, it would be judged as correct, if else, it would be judged as incorrect.

The common jargon used for competitive programming (regarding submission and judging solutions) will be as following.

- Checker: a program that check if your program is correct. For most cases, checker will compare your output with a set output file character by character.
- Verdict: the result of your submission, given from the judge. Common verdicts are as following.
  - AC: Accepted (or all correct), meaning that you have solved the problem. This is what you want to get.
  - CE: Compile error, caused by your program not compiling.
  - RTE: Runtime Error, caused by your program halting/crashing (exiting with nonzero return).
  - WA: Wrong answer, caused by your program terminating correctly, but with an incorrect output. (violating purpose 1)
  - TLE: Time limit exceeded, caused by your program taking more time than the alloted time for the problem (violating purpose 2)
  - MLE: Memory limit exceeded: caused by your program taking more memory than the alloted memory for the problem (violating purpose 3)

To familiarize yourself with the process of submitting and observing different verdicts, I recommend you to attempt solving some simple problems;

Some example problems would be

https://www.acmicpc.net/problem/1000 (Baekjoon online judge, 1000, Korean)

https://codeforces.com/problemset/problem/1772/A (Codeforces, 1772A, English)

Message

which are some variants of the problem of adding two numbers and outputting the result.

Implementation detail

Implementation details will normally not be covered in further content, since this project is meant to be language-independant: but I thought it would be beneficial to state some implementation details regarding inputting and outputting, at least for python and C++.

- In python, you take input that are separated by spaces using `map(int,input().split())`.
  - For instance, `a,b = map(int,input().split())` will input two integers separated, and `a = list(map(int,input().split()))` will input a list of integers separated by a space, and store it into `a`.
- In C++, you normally want to use `<iostream>` as it is easier to use than `<stdio.h>` (use `cin` and `cout`)
- You have to note that inputting and outputting are both very slow.
  - for python, you might want to use `sys.stdin.readline()` and `sys.stdout.write()` instead of `input()` and `print()`, for faster performance.
  - for C++, you might want to use `std::ios_base::sync_with_stdio(false); std::cin.tie(nullptr);` to untie iostream input/output for faster performance. Note that this is a source of error if you are solving an interractive problem.
- Interractive problems are rare type of problems where your program would have to interract (usually query) the checker program multiple times to solve the problem, usually within a set number of queries.
  - For both python and C++, you have to remember that you have to flush the output buffer every time you output for interractive problems: in python, this can be done with `sys.stdout.flush`, and in C++, this can be done with `cout << "message to output" << flush` (assuming that you are using `std` namespace)
- For further information regarding inputting and outputting in a specific format, consult language documentations and online sources, as it is moderately easy to find.

Common mistakes

- In some course you might have taken, you might have learned to output meaningful prompts for users to see and understand what to input. For instance, in python, something like using `input("Input the first integer here:")`. In most online judges and competitive programming, this will cause your solution to be judged as incorrect, since you are outputting lines that were not specified to input.
- Keeping the format strictly is always important: remove all trailing whitespaces (unless specified otherwise), separate each lines and end your output with a newline character `"\n"`, (unless specified otherwise), and so on. There are some checkers that do have some leniency in formatting - but do not assume such leniency, and always ensure that you have the exact same format as specified.

Summary

The general process of solving a problem (in competitive programming) is as following.

1. Read and understand the problem. (input/output format, what your program should do etc)
2. Write a program that outputs correctly given a valid input, according to the problem.
3. Submit to judge, and wait until you get the verdict.

Message

4. If AC, you have solved the problem. If not, find and fix errors within the program until you get AC.

Time and space

As your solution is only given limited time and space to work with, the analysis of time complexity and space complexity of different solutions and algorithms are extremely important.

Since the input size is variable for different test cases (bounded by a certain bound, given in the problem), for easy comparison between different methods, we often use asymptotic approximations. For the purpose of competitive programming, you would need some knowledge regarding the big-O, Theta, and big-omega notations. The usage and comparison of different time/space complexity, within UCI, is covered in 33, 46 and 161 in different levels:

https://en.wikipedia.org/wiki/Time_complexity (Wikipedia, Time complexity)

https://en.wikipedia.org/wiki/Space_complexity (Wikipedia, Space complexity)

these two articles are good articles to read regarding time complexity and space complexity in terms of theory.

In short, don't be too suprised to find usage of such notations in further content, when discussing about the time an operation takes / amount of memory an operation requires.

Examples of time complexity and space complexity

Since we will primarily use big-O notation, here are some examples of using it.

- Running a for loop on an array of length n: $O(n)$ time.
- Finding all permutations of an array of length n: $O(n!)$ time.
- Calculating $a + b$, where a and b are `int` values: $O(1)$ time.

for space complexity:

- Storing an integer: $O(1)$ space.
- Storing an array of integers of length n: $O(n)$ space.
- Storing a graph with V verticies with E edges: $O(V + E)$ space.

We will go over time definition of different mathmatical constructs (such as graphs) later on - it is okay if you don't have a solid understanding on these topics yet.

Pragmatics

In general, we assume that a standard computer can make upto 10^9 simple operations in a second: and therefore, given a 1 second time limit…

$O(n)$ solution probably will run if $n \approxeq 10^7$, and will probably not run if $n \approxeq 10^{18}$.

$O(n^2)$ solution probably will run if $n \approxeq 10^3$, and will probably not run if $n \approxeq 10^6$.

Note that these are all "probably" since constants do matter: a $O(n \log n)$ solution with low constant might be faster than $O(n)$ with high constant, and so on. You would have to develop your own intuition regarding which solutions will run within given time - from experience.

Message